

MoMo Lens: Database Design Document

System: Mobile Money SMS Data Processing System

Course: Enterprise Web Development

Team Name: Web Artisans

Database Engine: MySQL

Project Board: MomoLens Project Board (GitHub) -

<https://github.com/users/kennywigamba/projects/2/views/2>

1. Project Overview

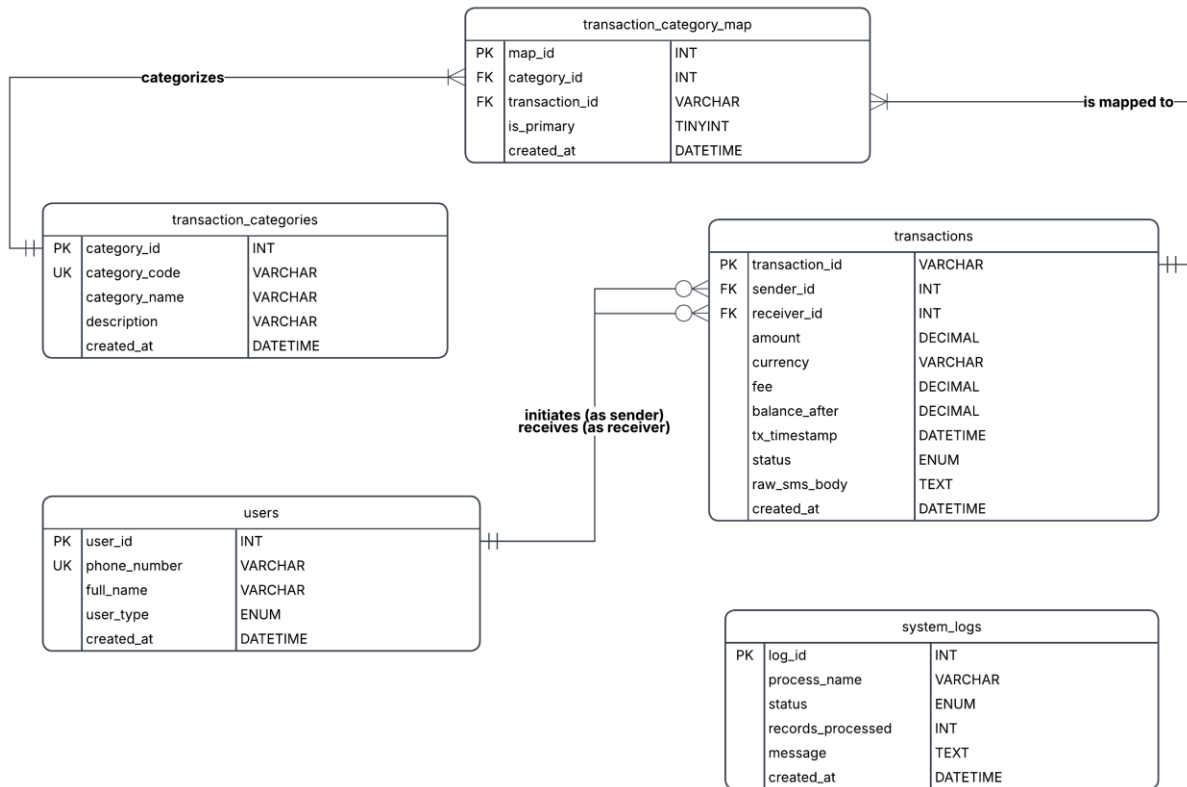
MoMo Lens processes Mobile Money (MoMo) SMS transaction data from MTN Rwanda. The system extracts financial records from text messages, cleans and categorizes them, stores them in a normalized relational database and exposes the data via JSON APIs for web dashboard analytics.

Our database design uses **5 tables** structured in Third Normal Form (3NF) to keep the data organized, avoid duplicate records and accurately represent real-world mobile money operations.

2. Entity Relationship Diagram (ERD) & Design Rationale

2.1 Visual ERD Diagram

The diagram below outlines the database schema, including primary keys, foreign keys, unique keys and relationship cardinalities:



2.2 Entity Relationships & Cardinalities

The schema uses the following relationships to model real-world mobile money workflows:

1. users to transactions (One-to-Many, Dual Relationship):

- **Sender (1 : N):** One user can initiate many outgoing transactions (users.user_id → transactions.sender_id).
- **Receiver (1 : N):** One user can receive many incoming transactions (users.user_id → transactions.receiver_id).
- Every transaction connects exactly one sender and one receiver.

2. transactions to transaction_categories (Many-to-Many via transaction_category_map):

- A single transaction can have multiple categories (for example, *Merchant Payment* and *Utility Bill*).
- A single category can apply to many transactions.

- This Many-to-Many relationship is resolved using the junction table `transaction_category_map`, which links `transactions.transaction_id` and `transaction_categories.category_id`.

3. **system_logs (Standalone Entity):**

- Operates independently with no foreign keys. It records data ingestion, XML parsing progress, and ETL error logs without putting locks on financial ledger tables.

2.3 Design Decisions & Rationale

1. **Single users Table for All Participants:** Instead of creating separate tables for senders and receivers, all participants (individuals, merchants, agents, banks) are stored in one `users` table. In mobile money, a user who receives money in the morning can send money in the afternoon. The `transactions` table links to `users` twice: once for `sender_id` and once for `receiver_id`. This prevents duplicate user profiles and keeps the schema normalized.
2. **Many-to-Many Relationship via `transaction_category_map`:** A transaction can belong to more than one category (for example, a Merchant Payment that also counts as a Utility Bill). Instead of adding multiple category columns to the `transactions` table, we use the junction table `transaction_category_map`. This allows flexible tagging and includes an `is_primary` flag to indicate the main category.
3. **Storing Raw SMS Text:** We store the original carrier SMS message in `transactions.raw_sms_body`. This preserves the original message for audit and verification without needing an extra join table.
4. **Performance & Integrity:** We placed indexes on frequently searched columns (`phone_number`, `tx_timestamp`, `category_code`) and added database-level CHECK constraints to ensure amounts, fees, and balances can never be negative.

3. Data Dictionary

3.1 Table: `users`

Stores all individuals, merchants, agents and institutions participating in transactions.

Column	Data Type	Nullable	Key	Default	Description
<code>user_id</code>	INT	NO	PK	AUTO_INC	Unique user ID.
<code>phone_number</code>	VARCHAR(20)	NO	UK	None	Unique phone number.
<code>full_name</code>	VARCHAR(100)	NO		None	Registered name of

					customer or merchant.
user_type	ENUM	NO		'INDIVIDUAL'	Role: 'INDIVIDUAL', 'MERCHANT', 'AGENT', 'BANK', 'SYSTEM'.
created_at	DATETIME	NO		CURRENT_TIMESTAMP	Registration timestamp.

3.2 Table: transaction_categories

Master list of supported transaction types.

Column	Data Type	Nullable	Key	Default	Description
category_id	INT	NO	PK	AUTO_INC	Unique category ID.
category_code	VARCHAR(30)	NO	UK	None	Short unique code (e.g. P2P_TRANSFER, MERCHANT_PAYMENT).
category_name	VARCHAR(100)	NO		None	Display name of category.
description	VARCHAR(255)	YES		NULL	Category description.
created_at	DATETIME	NO		CURRENT_TIMESTAMP	Creation timestamp.

3.3 Table: transactions

Primary financial ledger parsed from SMS messages.

Column	Data Type	Nullable	Key	Default	Description & Rules
transaction_id	VARCHAR(64)	NO	PK	None	Financial transaction ID from SMS.
sender_id	INT	NO	FK	None	Payer account (references users.user_id).

receiver_id	INT	NO	FK	None	Payee account (references users.user_id).
amount	DECIMAL(12,2)	NO		None	Transferred amount. CHECK (amount > 0).
currency	VARCHAR(3)	NO		'RWF'	Currency code.
fee	DECIMAL(10,2)	NO		0.00	Network fee. CHECK (fee >= 0).
balance_after	DECIMAL(12,2)	NO		None	Balance after transaction. CHECK (balance_after >= 0).
tx_timestamp	DATETIME	NO		None	Transaction date and time.
status	ENUM	NO		'COMPLETED'	'COMPLETED', 'PENDING', 'FAILED', 'REVERSED'.
raw_sms_body	TEXT	YES		NULL	Original SMS text for auditing.
created_at	DATETIME	NO		CURRENT_TIMESTAMP	Record creation timestamp.

3.4 Table: transaction_category_map (Junction Table)

Resolves the Many-to-Many relationship between transactions and categories.

Column	Data Type	Nullable	Key	Default	Description & Rules
map_id	INT	NO	PK	AUTO_INC	Unique mapping ID.
transaction_id	VARCHAR(64)	NO	FK	None	References transactions(transaction_id) ON DELETE CASCADE.
category_id	INT	NO	FK	None	References

					transaction_categories(category_id) ON DELETE RESTRICT.
is_primary	TINYINT(1)	NO		1	1 = primary category, 0 = secondary tag.
created_at	DATETIME	NO		CURRENT_TIMESTAMP	Creation timestamp.
Composite	-	-	UK	-	UNIQUE KEY (transaction_id, category_id) prevents duplicate tags.

3.5 Table: system_logs

Audit log for data parsing and ETL batch jobs.

Column	Data Type	Nullable	Key	Default	Description
log_id	INT	NO	PK	AUTO_INC	Unique log ID.
process_name	VARCHAR(50)	NO		None	Script or module name (e.g. XML_PARSER, DB_LOADER).
status	ENUM	NO		None	'SUCCESS', 'WARNING', 'ERROR'.
records_processed	INT	NO		0	Number of records processed.
message	TEXT	YES		NULL	Status or error message.
created_at	DATETIME	NO		CURRENT_TIMESTAMP	Log timestamp.

4. Security & Data Integrity Rules

4.1 Financial CHECK Constraints

Enforced on the transactions table to prevent invalid numbers:

```
CONSTRAINT chk_amount_pos CHECK (amount > 0),  
CONSTRAINT chk_fee_non_neg CHECK (fee >= 0),  
CONSTRAINT chk_balance_non_neg CHECK (balance_after >= 0)
```

Testing Negative Balance (Rejected by MySQL):

```
INSERT INTO transactions (transaction_id, sender_id, receiver_id, amount, currency,  
fee, balance_after, tx_timestamp)  
VALUES ('FAIL_TEST_01', 1, 2, 500.00, 'RWF', 0.00, -150.00, NOW());
```

MySQL Output:

ERROR 3819 (HY000): Check constraint 'chk_balance_non_neg' is violated.

Testing Negative Amount (Rejected by MySQL):

```
INSERT INTO transactions (transaction_id, sender_id, receiver_id, amount, currency,  
fee, balance_after, tx_timestamp)  
VALUES ('FAIL_TEST_02', 1, 2, -500.00, 'RWF', 0.00, 1000.00, NOW());
```

MySQL Output:

ERROR 3819 (HY000): Check constraint 'chk_amount_pos' is violated.

4.2 Referential Integrity Rules

- **ON DELETE RESTRICT on users:** Prevents deleting a user if they are linked to existing transactions, protecting audit history.
- **ON DELETE CASCADE on transaction_category_map:** If a transaction is deleted, its category mapping tags are automatically removed.
- **UNIQUE KEY uq_tx_cat:** Prevents attaching the same category multiple times to the same transaction.

5. Sample Queries & Actual Results

Executed on MySQL (momo_lens_db).

Query 1: Complete Financial Ledger

Joins transactions with sender, receiver, and primary category:

```
SELECT  
    t.transaction_id,  
    s.full_name AS sender_name,  
    r.full_name AS receiver_name,
```

```

    t.amount,
    t.fee,
    t.balance_after,
    c.category_name AS primary_category,
    t.tx_timestamp,
    t.status
FROM transactions t
JOIN users s ON t.sender_id = s.user_id
JOIN users r ON t.receiver_id = r.user_id
JOIN transaction_category_map m ON t.transaction_id = m.transaction_id AND
m.is_primary = 1
JOIN transaction_categories c ON m.category_id = c.category_id
ORDER BY t.tx_timestamp DESC;

```

Output:

transaction_id	sender_name	receiver_name	amount	fee	balance_after	primary_category	tx_timestamp	status
13947831685	Account Owner	PayPal PAYMENT LTD	25000.00	250.00	40930.00	Merchant Payment	2026-09-14 10:10:00	COMPLETED
17818959211	Account Owner	Sam Kalisa	2000.00	100.00	66180.00	Merchant Payment	2026-09-13 16:45:00	COMPLETED
73214484437	Account Owner	Jane Uwera	1000.00	100.00	68280.00	Merchant Payment	2026-09-12 14:20:00	COMPLETED
51732411227	Account Owner	Sam Kalisa	600.00	20.00	69380.00	Merchant Payment	2026-09-11 11:30:00	COMPLETED
76662021700	Jane Uwera	Account Owner	20000.00	0.00	70000.00	Peer-to-Peer Transfer	2026-09-10 09:15:00	COMPLETED

Query 2: Multiple Categories per Transaction

Demonstrates the Many-to-Many relationship using GROUP_CONCAT:

```

SELECT
    t.transaction_id,
    t.amount,
    t.currency,
    GROUP_CONCAT(c.category_name ORDER BY m.is_primary DESC SEPARATOR ' | ') AS
categories
FROM transactions t
JOIN transaction_category_map m ON t.transaction_id = m.transaction_id
JOIN transaction_categories c ON m.category_id = c.category_id
GROUP BY t.transaction_id, t.amount, t.currency;

```

Output:

transaction_id	amount	currency	categories
13947831685	25000.00	RWF	Merchant Payment Utility Bill
17818959211	2000.00	RWF	Merchant Payment
51732411227	600.00	RWF	Merchant Payment
73214484437	1000.00	RWF	Merchant Payment
76662021700	20000.00	RWF	Peer-to-Peer Transfer

(Notice: Transaction 13947831685 belongs to both Merchant Payment and Utility Bill)

Query 3: Daily Summary Aggregation

Groups transactions by date and category:

```

SELECT
    DATE(t.tx_timestamp) AS tx_date,
    c.category_name,
    COUNT(t.transaction_id) AS total_count,
    SUM(t.amount) AS total_volume_rwf,
    SUM(t.fee) AS total_fees_rwf
FROM transactions t
JOIN transaction_category_map m ON t.transaction_id = m.transaction_id AND
m.is_primary = 1
JOIN transaction_categories c ON m.category_id = c.category_id
GROUP BY DATE(t.tx_timestamp), c.category_name;

```

Output:

tx_date	category_name	total_count	total_volume_rwf	total_fees_rwf
2026-09-10	Peer-to-Peer Transfer	1	20000.00	0.00
2026-09-11	Merchant Payment	1	600.00	20.00
2026-09-12	Merchant Payment	1	1000.00	100.00
2026-09-13	Merchant Payment	1	2000.00	100.00
2026-09-14	Merchant Payment	1	25000.00	250.00

6. JSON API Data Modeling

To provide clean API responses, foreign key IDs are resolved into nested objects. Below is a serialized transaction payload from `examples/complex_transaction.json`:

```
{
  "transaction_id": "13947831685",
  "sender": {
    "user_id": 1,
    "phone_number": "250788110381",
    "full_name": "Account Owner",
    "user_type": "INDIVIDUAL",
    "created_at": "2026-09-01T08:00:00Z"
  },
  "receiver": {
    "user_id": 5,
    "phone_number": "250795963036",
    "full_name": "PayPal PAYMENT LTD",
    "user_type": "MERCHANT",
    "created_at": "2026-09-01T08:30:00Z"
  },
  "amount": 25000.00,
  "currency": "RWF",
  "fee": 250.00,
  "balance_after": 40930.00,
  "tx_timestamp": "2026-09-14T10:10:00Z",
  "status": "COMPLETED",
  "categories": [
```

```

{
  "category_id": 2,
  "category_code": "MERCHANT_PAYMENT",
  "category_name": "Merchant Payment",
  "description": "Payments made to merchant codes",
  "is_primary": true
},
{
  "category_id": 5,
  "category_code": "UTILITY_BILL",
  "category_name": "Utility Bill",
  "description": "Payments for water, power, or utilities",
  "is_primary": false
}
],
"raw_sms_body": "TxId: 13947831685. Payment of 25,000 RWF to PayPal PAYMENT LTD
completed. Fee: 250 RWF. New balance: 40,930 RWF.",
"created_at": "2026-09-14T10:10:05Z"
}

```

7. Team Contributions

- **Ishimwe Rwigamba Kenny Louange (@kennyrwigamba):** Database architecture, SQL DDL/DML, constraints, live MySQL query testing.
- **Maunice Akaliza (@Akaliza-ux):** ERD design, JSON schemas & modeling, data dictionary documentation.

Tasks and progress tracked on the MomoLens Scrum Board

(<https://github.com/users/kennyrwigamba/projects/2/views/2>).